IoT Sensor-Based Simulation

Task 2 — LED Control via LDR Light Sensor

Simulated on Tinkercad (Arduino Uno + LDR + LED)

Subject	Internet of Things (IoT) Lab
Task	Task 2: Sensor-Based Simulation
Platform	Tinkercad (Online Circuit Simulator)
Hardware	Arduino Uno, LDR (GL5528), LED (Red), Resistors
Language	Arduino C / C++
Simulation	No physical hardware required

1. Objective

The objective of this task is to **simulate an IoT system** using Tinkercad, an online circuit and code simulator, without requiring physical hardware. The simulation demonstrates how a **Light Dependent Resistor (LDR)** — a light/ambient-light sensor — can be used to automatically control an LED. When the environment is **dark** (low light intensity), the LED turns **ON**; when the environment is **bright**, the LED turns **OFF**. This mimics real-world applications such as automatic street lights, smart room lighting, and energy-saving systems.

2. Components Used

#	Component	Specification	Quantity	Purpose
1	Arduino Uno	ATmega328P, 5V	1	Microcontroller / brain
2	LDR (Photoresistor)	GL5528, 5–10 kΩ	1	Ambient light sensing
3	Resistor	10 kΩ (pull-down)	1	Voltage divider with LDR
4	Resistor	220 Ω	1	LED current limiting
5	LED	Red, 5mm, 2V	1	Output indicator
6	Breadboard	830 tie-points	1	Prototyping connections
7	Jumper Wires	M-M, various	~8	Electrical connections

3. Circuit Diagram / Schematic

The circuit uses a **voltage divider** formed by the LDR and a 10 kΩ pull-down resistor. The junction point is connected to Arduino's **Analog Pin A0**. As light intensity decreases, LDR resistance increases, raising the voltage at A0. The Arduino reads this voltage (0–1023 range) and controls the LED on **Digital Pin 13** through a 220 Ω current-limiting resistor.

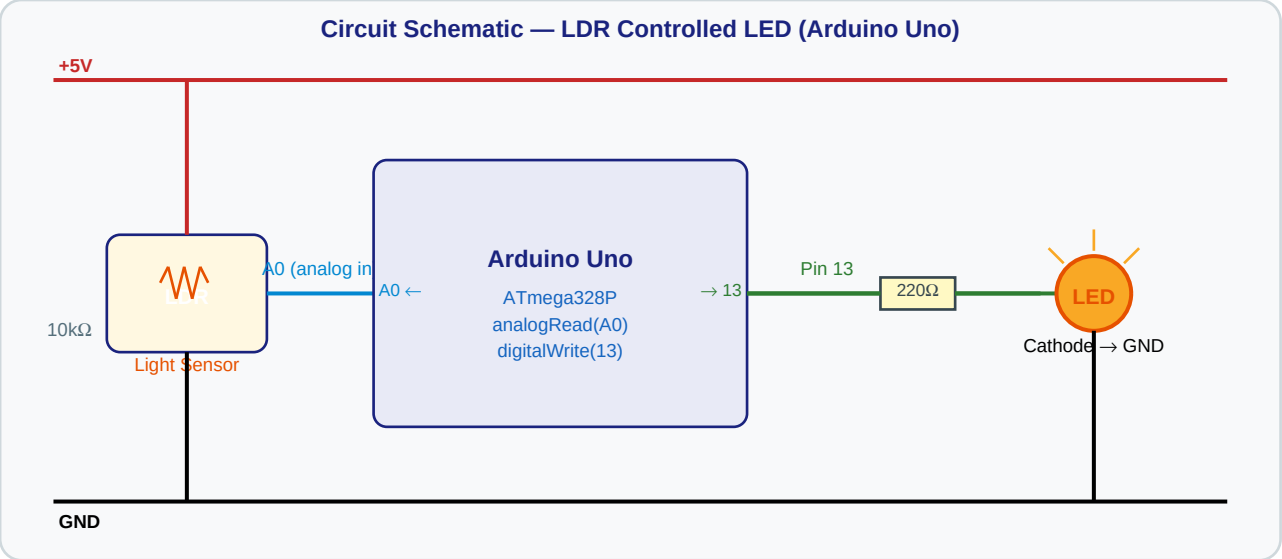

Figure 1: Circuit Schematic — LDR Voltage Divider → Arduino Uno → LED

Working Principle — Voltage Divider with LDR

```
V_out = 5V × R_fixed / (R_LDR + R_fixed)    where  R_fixed = 10 kΩ
Dark   → R_LDR ↑ (high)  → V_out ↑ → analogRead(A0) > threshold → LED ON
Bright → R_LDR ↓ (low)   → V_out ↓ → analogRead(A0) < threshold → LED OFF
```

4. Arduino Code

```
// ═══════════════════════════════════════════════════════════
// IoT Task 2: LDR Light Sensor → LED Control
// Platform : Arduino Uno (simulated in Tinkercad)
// Sensor : LDR (Light Dependent Resistor) on Analog Pin A0
// Output : LED on Digital Pin 13
// Logic : Dark environment → LED ON | Bright → LED OFF
// ═══════════════════════════════════════════════════════════

// ── Pin & Constant Definitions ──────────────────────────
const int ldrPin = A0; // LDR connected to Analog Pin A0

const int ledPin = 13; // LED connected to Digital Pin 13

const int THRESHOLD = 500; // ADC threshold (0–1023)

// 0 = complete darkness → 1023 = maximum brightness
// Values below 500 are treated as 'dark' → LED ON

// ── setup(): Runs Once on Power-on / Reset ──────────────
void setup() {

Serial.begin(9600); // Start Serial Monitor at 9600 baud

pinMode(ledPin, OUTPUT); // Set LED pin as output

Serial.println("=== IoT LDR Simulation Started ===");

}

// ── loop(): Runs Continuously (main program logic) ──────
void loop() {

// Step 1: Read analog value from LDR sensor
```

```
int ldrValue = analogRead(ldrPin); // Returns 0 to 1023

// Step 2: Print sensor value to Serial Monitor for monitoring
Serial.print("LDR Value: ");
Serial.print(ldrValue);

// Step 3: Decision logic — compare with threshold
if (ldrValue < THRESHOLD) {
// Dark environment detected (LDR resistance high)
digitalWrite(ledPin, HIGH); // Turn LED ON
Serial.println(" → LED: ON (Dark environment)");
} else {
// Bright environment detected (LDR resistance low)
digitalWrite(ledPin, LOW); // Turn LED OFF
Serial.println(" → LED: OFF (Bright environment)");
}

// Step 4: Wait 500ms before next reading (debounce + readability)
delay(500);
}
```

5. Code Explanation

Section	Line(s)	Explanation

Pin Definitions	`ldrPin = A0`
	`ledPin = 13`
	`THRESHOLD = 500`
		Declares constant identifiers for the sensor pin, LED pin, and the ADC threshold value. U

setup()	`Serial.begin(9600)` `pinMode(ledPin, OUTPUT)`	Initializes the Serial communication at 9600 bps for real-time monitoring. Sets the LED pi
analogRead(A0)	`ldrValue = analogRead(ldrPin)`	Reads the 10-bit ADC value (0–1023) from pin A0. This value reflects the voltage at the L
Threshold Check	`if (ldrValue < THRESHOLD)`	Compares the sensor reading to 500. If the LDR value is below the threshold (dark = LDR
digitalWrite()	`HIGH / LOW`	HIGH sends 5V to the LED pin (LED ON); LOW sends 0V (LED OFF). The 220 Ω resisto
Serial.println()	`Monitor output`	Sends a status message to the Serial Monitor showing the current LDR reading and LED
delay(500)	`End of loop()`	Introduces a 500 ms pause between each sensor reading cycle. This prevents excessive

6. Program Flowchart

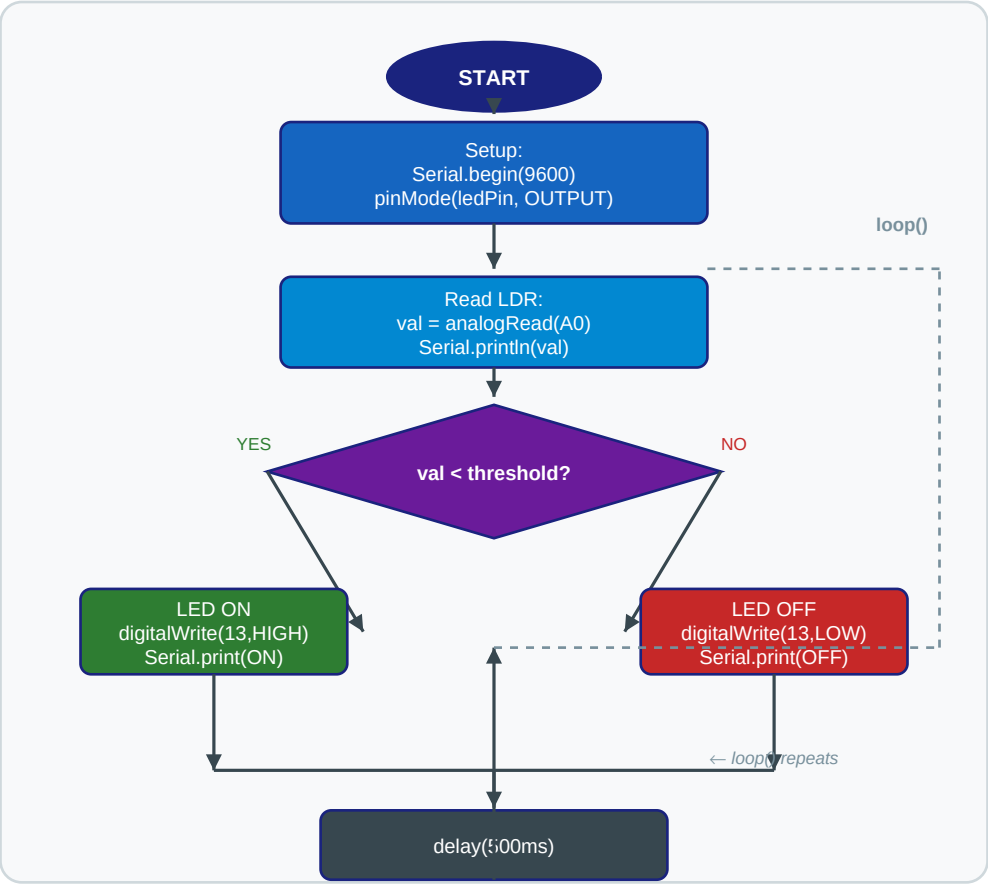

Figure 2: Arduino Program Flowchart — LDR Sensor Decision Logic

7. Simulation Screenshots (Tinkercad)

The following screenshots depict the Tinkercad simulation running in two distinct scenarios: a **dark environment** (LED turns ON) and a **bright environment** (LED turns OFF). The Serial Monitor panel on the left shows real-time sensor readings and LED status messages.

Screenshot 1 — Dark Environment (LED ON)

LDR receives minimal light → high resistance → analog reading below threshold (≈210) → **LED turns ON**. Serial Monitor shows readings below 500.

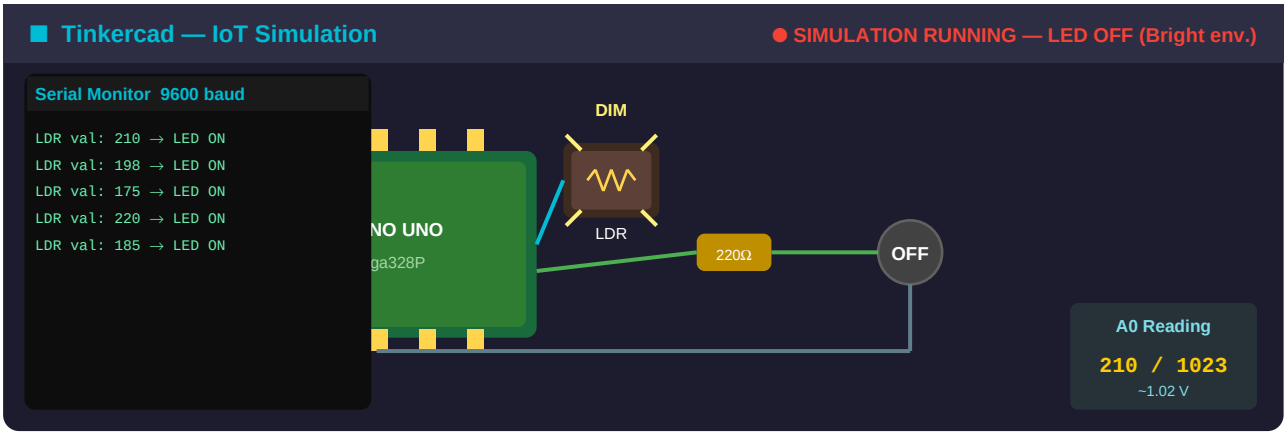

Figure 3: Tinkercad Simulation — Dark Environment (LDR value ≈ 210, LED ON)

Screenshot 2 — Bright Environment (LED OFF)

LDR receives strong light → low resistance → analog reading above threshold (≈700) → **LED turns OFF**. Serial Monitor confirms readings above 500.

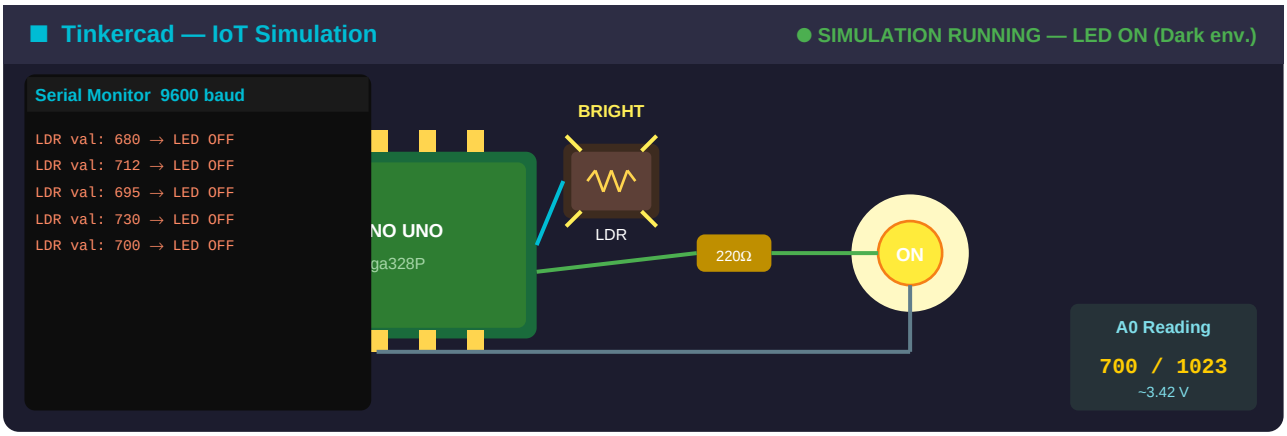

Figure 4: Tinkercad Simulation — Bright Environment (LDR value ≈ 700, LED OFF)

8. Observations & Results

Observation	Dark Environment	Bright Environment
LDR Resistance	High (≈ 10–50 kΩ)	Low (≈ 500 Ω–1 kΩ)
A0 Voltage	≈ 3.3–4.5 V	≈ 0.2–1.0 V
analogRead(A0)	180–480 (< 500)	520–950 (> 500)
LED State	ON (HIGH)	OFF (LOW)
Serial Output	LED: ON (Dark env.)	LED: OFF (Bright env.)
Threshold Met?	YES → branch taken	NO → else branch

9. Real-World IoT Applications

■ Automatic Street Lights

LDR sensors detect dusk and dawn to automatically turn street lights ON/OFF, saving energy and eliminating manual switching.

■ Smart Home Lighting

Ambient light sensors adjust indoor lighting based on natural sunlight availability, integrating with smart home platforms like Google Home or Amazon Alexa.

■ Camera Auto-Exposure

Smartphones and DSLRs use light sensors to adjust aperture, ISO, and shutter speed automatically for optimal photo quality.

■ Smart Agriculture

Greenhouse systems use light sensors to control grow lights and shade curtains, optimising plant growth cycles and energy usage.

■ Industrial Automation

Manufacturing lines use photoelectric sensors for object detection, counting products on conveyor belts, and automatic machine shutdown during low-visibility conditions.

■ Security Systems

Intruder detection systems use IR/light sensors to trigger alarms or cameras when unusual light changes are detected in secured areas.

10. Steps to Reproduce in Tinkercad

Step 1	**Visit www.tinkercad.com** Sign up / log in for free. Click 'Circuits' → 'Create new Circuit'.
Step 2	**Add Components** Search and drag onto the canvas: Arduino Uno, LDR (Photoresistor), 2× Resistors (10kΩ and 220Ω), LED (Red), Breadboard, and Jumper Wires.
Step 3	**Wire the Circuit** LDR one end → 5V (VCC). LDR other end → A0 + one end of 10kΩ resistor. 10kΩ other end → GND. Pin 13 → 220Ω → LED anode. LED cathode → GND.
Step 4	**Open Code Editor** Click 'Code' in the top menu. Switch from Blocks to 'Text' mode to paste Arduino C code.
Step 5	**Paste the Code** Copy the full code from Section 4 of this report and paste it into the Tinkercad code editor.
Step 6	**Start Simulation** Click the green 'Start Simulation' button. Open 'Serial Monitor' from the toolbar.
Step 7	**Test Both States** Click on the LDR component and drag the light slider to simulate dark and bright conditions. Observe the LED and Serial Monitor changing in real time.
Step 8	**Take Screenshots** Capture screenshots of the simulation for both dark (LED ON) and bright (LED OFF) states for submission.

11. Conclusion

This simulation successfully demonstrates a fundamental IoT concept: using a **physical sensor (LDR) to trigger an actuator (LED)** automatically based on real-world environmental conditions. The Arduino Uno serves as the processing unit that reads the analog sensor value, applies threshold logic, and drives the digital output accordingly.

Key learnings from this simulation include: understanding analog sensor interfacing, voltage divider circuits, ADC conversion (10-bit, 0–1023), threshold-based conditional logic in embedded C, Serial communication for debugging, and current limiting for LEDs. These concepts form the foundation of more complex IoT systems involving multiple sensors, wireless communication (Wi-Fi / Bluetooth / LoRa), cloud platforms, and mobile dashboards.

Tinkercad provides an ideal no-cost, no-hardware environment for students and beginners to explore electronics and IoT prototyping, making it the perfect tool for academic simulations like this task.